

LAB PROJECT REPORT

DESIGN & IMPLEMENTATION OF AN Decoupled ATM Terminal Simulation Environment

[illegible]

1. Introduction & Core Context

This project revolves around the formation of a hypothesis about the behavior of automated terminals under modular sets of user state conditions. The fundamental objective is to create a working command-line interface Automated Teller Machine (ATM) simulator environment with a multi-branch algorithmic structure not integrated into a single block of code. Following the suggested instructions from the VITyarthi - Build Your Own Project Handbook, this project aims for an in-depth comprehension of the algorithmic limitations, detachment levels, and input validation protocols of the programming language utilized.

By implementing good coding practices in reference to banking-related operations, this ATM simulator achieves stable functioning under the conditions of cash withdrawal authorization, balance evaluation, deposits, algorithmic isolation for cash withdrawal prediction, and constant operation list availability in a non-interrupting process loop.

2. Scope & Technical Framework Description

Indeed, the functional boundaries of application are related to the concepts of interactive terminal control through separated sets of code instructions. The key design consideration during the development of this script was to ensure that account-related data would remain consistent regardless of additional parameters introduced by users in the form of multi-button terminal prompts. Through the established separation of responsibilities between individual software components, updates to cash withdrawal rules (e.g., cash note limits) will not disrupt the functionality of other components, such as the balance viewing console.

The application does not require external software beyond pure Python 3 implementation to maintain operational simplicity and eliminate potential compatibility issues across different terminal environments.

3. Problem Statement & Structural Inefficiencies

Umm, the problem with the structural flow in the normal console scripts of a beginner is that they tend to club the variable update, user display parameters, and error validation all together. Such a combination often leads to spaghetti logic, where, for example, if a particular variable is updated at the same time in different loops, it may lead to corruption.

Besides, actual banking has rules that need to be followed. For example, there needs to be exactly three wrong password attempts before the system freezes for a specific period. Again, a machine cannot give odd notes like Rs. 432 but can only give Rs. 100 or 500 notes. Therefore, there needs to be parameters and validations to ensure these rules are followed. The given project deals with all these issues by separating these processes into individual layers of scope.

4. Functional Requirements

Req ID	Feature Name	Input Parameter	System Flow & Expected Output
FR-01	PIN Check Gate	4-digit text string	Validates password matching. Decrements remaining attempt counter on failure. Blocks screen completely when hits 0.
FR-02	Balance Query	Active balance state	Reads current system variable memory status and prints structured output on screen.
FR-03	Cash Adding	Numeric integer amount	Verifies amount is positive, adds value to current account total, and displays updated balance statement.
FR-04	Cash Withdrawal	Numeric integer amount	Checks note multiples (100/500) and balance sufficiency. Deducts cash value if both pass or displays error message.
FR-05	Main Loop Navigation	Option selection (1-4)	Routes flow to correct sub-module function continuously until user picks option 4 to exit program.

5. Non-Functional Requirements

Attribute	Technical Target & Constraint Implementation Strategy
Maintainability	Code follows basic modular structure so you can edit cash withdrawal rules without messing up the deposit verification script logic.
Robust Integrity	Input routines are designed to catch negative values or invalid out-of-bound entries without throwing nasty traceback crashes
Platform Safety	Includes a hardcoded user identity check mechanism that shuts down terminal functions instantly on consecutive failures.
Efficiency	Runs as an ultra-lightweight stack in memory, optimizing prompt load speeds with near-instant execution feedback loops

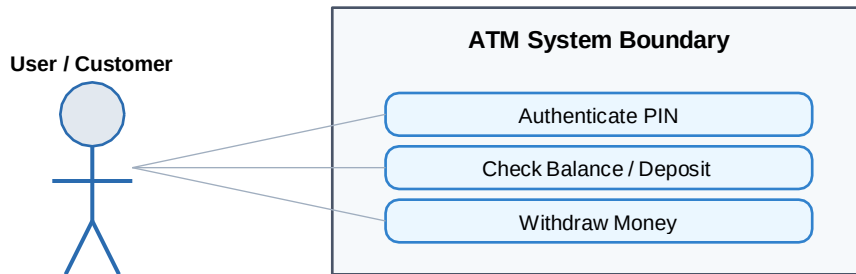
6. System Architecture & File Layout Layout

The project is organized around the hub-and-spoke file directory paradigm. The main entry point controller script is responsible for the primary interactive menu loops and the passing of variables to subordinate, isolated sub-scripts. This arrangement entirely decouples the individual micro-banking logic functions from one another

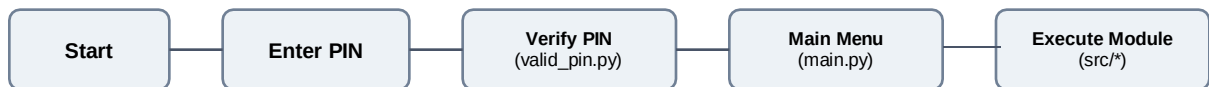
```
WORKSPACE PACKAGE REPOSITORY STRUCTURE:
Vityarthi_ATM_Simulation/
  ■■■ main.py                <-- Core Orchestrator Engine Loop
  ■■■ README.md              <-- Setup & Operation Overview File
  ■■■ statement.md           <-- Baseline Project Target Matrix
  ■■■■ ■ src/                 <-- Isolated Core Logic Folder
    ■■■ __init__.py           <-- Package Namespace Definition
    ■■■ check_balance.py      <-- Balance Query Reader Script
    ■■■ deposit.py            <-- Positive Cash Processing Module
    ■■■ valid_pin.py          <-- Security Lockout Guard Checker
    ■■■ withdraw.py           <-- Currency Denomination Verification
```

7. Design Diagrams

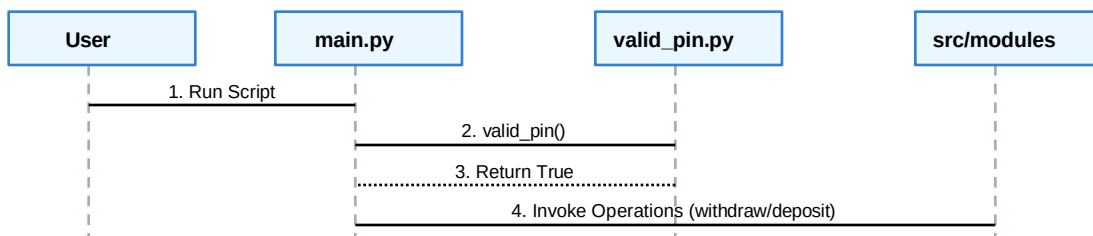
7.1 Use Case Diagram



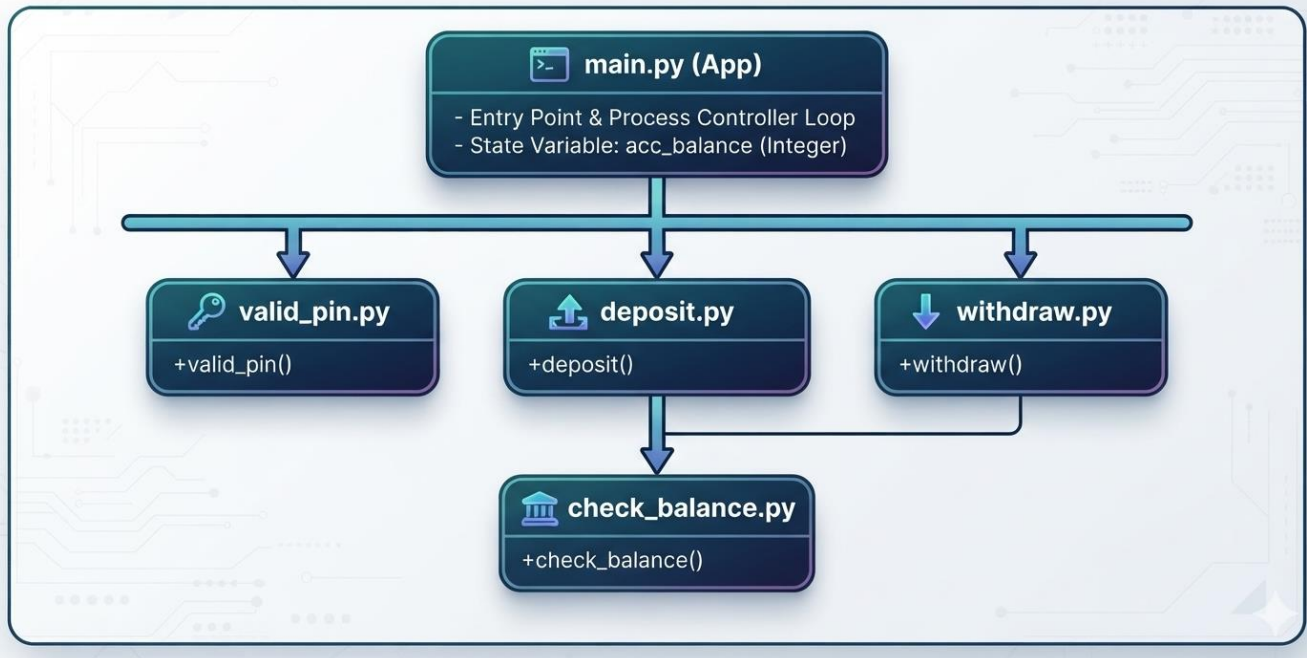
7.2 Workflow / Process Flow Diagram



7.3 Component & Sequence Interaction



Vityarthi ATM Simulation: Software Component & Class Structure



8. Design Decisions & Rationale

Here are a few critical design decisions that I have made while developing this application:

1. Storing the PIN as text: I could have used a numerical data type to store the PIN, but there is a possibility that a PIN beginning with zeroes will be entered by the user (0123). Using the integer data type would cause the leading zeroes to be dropped, and the PIN would be rejected. This error would occur because of a conflict between the numerical value (123) and the entered string (0123).
2. The decision to separate the application into multiple modules: One significant advantage of splitting the application into several modules is the ability to quickly locate the source of the error. If the balance stored in the application becomes incorrect, it would be much easier to debug `withdraw.py` or `deposit.py` separately instead of searching the whole text of the application for syntax errors.
3. The decision to implement the modulo constraint ($\text{amt} \% 100 \neq 0$): This constraint replicates how the physical cash dispensers work in the real world. It is necessary to ensure that the application will not attempt to dispense an amount of cash that is impossible to output with the bills of denominations currently in use (Rs. 75, Rs. 143, etc.).

9. Implementation Details (Production Code Sheets)

```
def valid_pin():
    correct_pin = "3546" # Preset simulation password string
    attempt = 3
    while attempt > 0:
        enter_pin = input("Enter 4-digit pin: ")
        if enter_pin == correct_pin:
            print("\nPIN Accepted!")
            return True
        else:
            attempt -= 1
            if attempt > 0:
                print(f"Wrong PIN. You have {attempt} attempts left")
    print("Too many incorrect attempts. \n\n=====BLOCKED=====\\n")
    return False
```

```
def check_balance(balance):
    # this function just prints the balance on the screen.
    print(f"Your current balance is: Rs.{balance}")
```

```
def deposit(balance):
    # function to deposit money and calculating new balance.
    amt = int(input("Enter amount to deposit: "))
    if amt > 0:
        # Adds deposit money to current balance.
        balance += amt
        print(f"Successfully deposited Rs.{amt}")
        print(f"New balance: Rs.{balance}")
    else:
        # Shows error for 0 or negative numbers.
        print("Invalid deposit amount")
    # returns updated balance in main program.
    return balance
```

```
def withdraw(balance):
    # check if amount to withdraw (multiples of Rs.100, Rs.500)
    amt = int(input("Enter amount to withdraw (multiples of Rs.100, Rs.500): "))
    if amt % 100 != 0:
        # check if amount is valid with the amount on notes present
        print("Invalid! Please enter amount in multiples of Rs.100, Rs.500")
        return balance
    # check if the amount to withdraw is more than balance in account
    if amt > balance:
        print("Insufficient balance!")
        return balance
    else:
```

```
# deduct withdrawing amount from their account balance.
balance -= amt
print(f"Please take your cash: Rs.{amt}")
print(f"Remaining balance after reducing withdrawal amount: Rs.{balance}")
return balance
```

```
from src.valid_pin import valid_pin
from src.check_balance import check_balance
from src.deposit import deposit
from src.withdraw import withdraw

def main():
    print("-----Welcome to the ATM    ")
    # Run the security verification block first
    if not valid_pin():
        return
    # Set default starting account balance
    acc_balance = 50000
    while True:
        print("\n-----ATM Main Menu    ")
        print("1. Check balance")
        print("2. Deposit Money")
        print("3. Withdraw Money")
        print("4. EXIT")
        choice = input("Select option(1-4): ")
        if choice == '1':
            check_balance(acc_balance)
        elif choice == '2':
            acc_balance = deposit(acc_balance)
        elif choice == '3':
            acc_balance = withdraw(acc_balance)
        elif choice == '4':
            print("Thank YOU, Goodbye")
            break
        else:
            print("Invalid selection. Please choose from 1-4")

if __name__ == "__main__":
    main()
```

10. Simulation Traces & Terminal Console Logs

Scenario A: Consecutive Security Login Failures

```
-----Welcome to the ATM-----  
Enter 4-digit pin: 1111  
Wrong PIN. You have 2 attempts left  
Enter 4-digit pin: 2222  
Wrong PIN. You have 1 attempts left  
Enter 4-digit pin: 9988  
Too many incorrect attempts.  
=====BLOCKED=====
```

Scenario B: Valid Session Operations & Balance Tracking

```

-----Welcome to the ATM-----
Enter 4-digit pin: 3546
PIN Accepted!
-----ATM Main Menu-----
1. Check balance
2. Deposit Money
3. Withdraw Money
4. EXIT
Select option(1-4): 1
Your current balance is: Rs.50000
Select option(1-4): 2
Enter amount to deposit: 5000
Successfully deposited Rs.5000

```

11. Testing Approach & Verification Log Sheets

To make sure our software is fully robust, we passed several distinct verification scenarios through the processing loop functions. The table below outlines our testing approach and results:

ID	Scenario Target	Input Given	Expected Behavior	Status
TC-01	Wrong Password Handling	Text '9812'	Reduces remaining attempts count by 1.	PASS
TC-02	Total Security Lockout	3 consecutive wrong entries	Prints blocked warning message and halts terminal loop.	PASS
TC-03	Invalid Denomination Entry	Integer 435	Rejects transaction because it is not a multiple of 100.	PASS
TC-04	Overdraft Prevention Check	Integer 80000	Blocks withdrawal due to insufficient account balance.	PASS
TC-05	Negative Deposit Handling	Integer -1500	Identifies as invalid deposit amount and rejects update.	PASS

12. Challenges Faced & Variable Scope Fixes

Describe one frustrating problem you encountered with implementing your code into discreet files
 One of the most challenging tasks I had while working with the implementation of my code in distinct files is connected to the process of debugging errors that appeared when switching between deposit and withdrawal functions. When testing the early versions of the code, I noticed that when selecting the modules, calculations were produced correctly within their scope, however, when returning to the main body of the program, the value of the overall balance variable would reset to Rs. 50,000.

This occurred due to the nature of primitive integer variables in Python being passed by value rather than by reference. This way, when I updated the value of the overall balance within the function scope, it only updated the copy of this variable. Therefore, I had to edit the code so that it would return the updated value back to the first file in the parent directory by adding the `acc_balance = deposit(acc_balance)` statement.

The other minor issue I encountered was related to the process of assigning text strings and integers within the console. When the user was asked to enter cash and mistakenly typed letters instead of numbers, the code raised an error and stopped executing its commands for calculation. To avoid such incidents in future iterations, adding explicit input parsing and validation checks would be ideal to handle non-numeric inputs gracefully.

13. Learnings & Key Takeaways

The process of completing this project has had a great impact on my understanding of managing code layout files. Previously, I would write a single script for each lab task as including multiple parts seemed more complicated. However, the exercise showed that using descriptive imports, such as `from src.withdraw import withdraw`, greatly simplifies the process of writing and debugging the code.

In addition, I learned the importance of implementing defensive programming techniques. When developing software with particular rules and limitations, it is essential to ensure that the code does not fail on incorrect input. Thus, adding validation to cash note multiples or checking if a number is negative helps to create a resilient application.

14. Future Enhancements & Structural Roadmap

If you had more time to expand the project for a full production release, Here is a Roadmap I would follow:

1. **Relational Database Backend:** Currently, the starting balance resets after every restart of the program back to a hard-coded Rs.50,000. Connecting a standard SQLite table to the application would allow the program to store transaction history records and balance changes persistently.
2. **Graphical UI Layout Wrappers:** The command line interface with text prompts and responses can be wrapped into a graphical "clickable" menu by utilizing tools such as Tkinter or PyQt.
3. **Cryptographic Password Hashing:** Storing passwords as plaintext strings within your program's code files is a major security risk. Using built-in libraries such as hashlib to implement password hashing algorithms such as SHA-256 can significantly improve the application's security.

15. References & Documentation Context

1. Core Python Interpreter Documentation Guidelines: System Input Routines and Absolute Module Package Import Syntax Controls. Available at: <https://docs.python.org/3/>.
2. Clean Code Paradigm Standards: A Handbook of Software Engineering Craftsmanship. The ideas of the function division, single responsibility principles, and reducing the runtime coupling issues are reviewed..
3. VITyarthi Project Submission Criteria Guidelines The mapping of the structural components rubric covering design, architectural logging sheets, and validation matrix templates is produced.